Systemsoftware WiSe 2026/27 Übung 1

Philipp Grommes 1538901

Aufgabe 1

Ein Programm ist eine Abfolge gespeicherter Anweisungen, welche festlegen, was ein Computer ausführen soll. Startet man ein Programm, so entsteht daraus ein Prozess. Ein Prozess ist also ein laufendes Programm. Jeder Prozess besitzt Ressourcen wie beispielsweise Arbeitsspeicher. Innerhalb eines Prozesses können einer oder mehrere Threads enthalten sein, welche verschiedene Aufgaben parallel ausführen können. Einen Thread kann man also als einen Ausführungsstrang innerhalb eines Prozesses bezeichnen. Threads teilen sich die Ressourcen des zugehörigen Prozesses, um an diesem zu arbeiten. Innerhalb eines Prozesses können sie sich die Arbeit aufteilen und parallel oder nacheinander ausgeführt werden.

Aufgabe 2

a)

Zusätzliche Kerne und eine höhere Taktung können einen Prozess zwar beschleunigen, jedoch nicht unendlich.
Ein Prozess kann nur dann von zusätzlichen Kernen profitieren, wenn seine Aufgaben parallel ausgeführt werden können. Manche Aufgaben innerhalb eines Prozesses müssen jedoch sequenziell bearbeitet werden und können so nicht auf mehrere Kerne verteilt werden. Dadurch bleibt der Zeitaufwand für den sequenziellen Teil immer gleich, egal wie viele Kerne hinzugefügt werden.
Durch die Erhöhung der Taktung, kann ein Prozessorkern mehr Rechenschritte pro Sekunde ausführen. Dies würde theoretisch funktionieren, ist allerdings durch technische Grenzen wie die Wärmeentwicklung praktisch nicht möglich.

b)

Amdahlsches Gesetz:

$$T_{MultiCore}(s,n) = T_{SingleCore}(s) + T_{SingleCore}\left(\frac{1-s}{n}\right)$$

2 Kerne:

$$T_{MultiCore}(0{,}75\ ,2) = 0{,}75s + \frac{0{,}25}{2}s = 0{,}875s$$

$$Speedup = \frac{1s}{0{,}875s} \approx 1{,}14$$

$$T_{MultiCore}(0{,}5\ ,2) = 0{,}5s + \frac{0{,}5}{2}s = 0{,}75s$$

$$Speedup = \frac{1s}{0{,}75s} \approx 1{,}33$$

$$T_{MultiCore}(0{,}75\ ,2) = 0{,}25s + \frac{0{,}75}{2}s = 0{,}625s$$

$$Speedup = \frac{1s}{0{,}625s} \approx 1{,}6$$

4 Kerne:

$$T_{MultiCore}(0{,}75\ ,4) = 0{,}75s + \frac{0{,}25}{4}s = 0{,}8125s$$

$$Speedup = \frac{1s}{0{,}8125s} \approx 1{,}23$$

$$T_{MultiCore}(0{,}5\ ,4) = 0{,}5s + \frac{0{,}5}{4}s = 0{,}625s$$

$$Speedup = \frac{1s}{0{,}625s} \approx 1{,}6$$

$$T_{MultiCore}(0{,}25\ ,4) = 0{,}25s + \frac{0{,}75}{4}s = 0{,}4375s$$

$$Speedup = \frac{1s}{0{,}4375s} \approx 2{,}29$$

8 Kerne:

$$T_{MultiCore}(0{,}75\ ,8) = 0{,}75s + \frac{0{,}25}{8}s = 0{,}78125s$$

$$Speedup = \frac{1s}{0{,}78125s} = 1{,}28$$

$$T_{MultiCore}(0{,}5\ ,8) = 0{,}5s + \frac{0{,}5}{8}s = 0{,}5625s$$

$$Speedup = \frac{1s}{0{,}5625s} \approx 1{,}78$$

$$T_{MultiCore}(0{,}25\ ,8) = 0{,}25s + \frac{0{,}75}{8}s = 0{,}34375s$$

$$Speedup = \frac{1s}{0{,}34375s} \approx 2{,}91$$

16 Kerne:

$$T_{MultiCore}(0{,}75\ ,16) = 0{,}75s + \frac{0{,}25}{16}s = 0{,}765625s$$

$$Speedup = \frac{1s}{0{,}765625s} \approx 1{,}31$$

$$T_{MultiCore}(0{,}5\ ,16) = 0{,}5s + \frac{0{,}5}{16}s = 0{,}53125s$$

$$Speedup = \frac{1s}{0{,}53125s} \approx 1{,}88$$

$$T_{MultiCore}(0{,}25\ ,16) = 0{,}25s + \frac{0{,}75}{16}s = 0{,}296875s$$

$$Speedup = \frac{1s}{0{,}296875s} \approx 3{,}37$$

Aufgabe 3

Betrachtete Testfälle:

3 Dateien, 30 Zeilen

100 Dateien, 2200 Zeilen

Für beide Testfälle wurden das sequenzielle Programm ***count_lines_in_files.py*** sowie die parallelisierte Variante ***count_lines_in_files_parallel.py*** getestet. Bei der parallelen Variante wurden jeweils 1, 2, 4, 8 und 16 Threads verwendet.

Bei beiden Testfällen war das ursprüngliche sequenzielle Programm schneller als die parallelisierte Variante.

Beim ersten Testfall mit lediglich 3 Dateien und 30 Zeilen führte bereits der Einsatz mehrerer Threads zu einer geringeren Effizienz. Der zusätzliche Verwaltungsaufwand für die Threads stand dabei in keinem sinnvollen Verhältnis zur geringen Menge an zu verarbeitenden Daten.

Auch beim zweiten Testfall mit 100 Dateien und 2.200 Zeilen konnte das parallele Programm das sequenzielle Programm nicht übertreffen. Innerhalb der parallelen Varianten zeigte sich jedoch, dass eine Erhöhung der Thread-Anzahl über 4 hinaus zu einer geringeren Effizienz führte.

Die Tests zeigen, dass eine Parallelisierung nicht automatisch zu einer schnelleren Ausführung führt. Durch die Verwaltung der Threads entsteht zusätzlicher Aufwand, der insbesondere bei kleineren Datenmengen zu sehen ist.

Außerdem führt eine höhere Anzahl an Threads nicht zwangsläufig zu einer besseren Performance. Ab einer bestimmten Thread-Anzahl kann der zusätzliche Aufwand die Effizienz sogar verschlechtern.

In meinen Tests war daher das sequenzielle Programm in beiden Fällen die schnellste Variante. Bei einer deutlich größeren Menge an zu verarbeitenden Daten könnte die parallelisierte Variante jedoch einen Geschwindigkeitsvorteil erzielen, da sich der zusätzliche Verwaltungsaufwand der Threads dann möglicherweise mehr lohnt.